Laravel Developer Assessment

Assessment Overview

Item	Details
Position	Laravel Developer
Duration	4 Hours
Framework	Laravel 11
Language	PHP 8.2+
Database	MySQL
Type	Backend API Assessment

Project Title

Smart Blood Inventory & Temperature Monitoring System

Develop a scalable backend API system for managing blood bags, refrigerator temperature monitoring, and inventory traceability.

The assessment is designed to evaluate:

- Laravel architecture knowledge
- Database design skills
- Authentication and authorization
- Mathematical and analytical logic
- Relationship handling
- Performance optimization
- Problem-solving and brainstorming capability
- Clean coding standards

Functional Requirements

1. Authentication & Authorization

Implement a secure authentication system using:

- Laravel Sanctum OR Laravel Passport

User Roles

- Admin
- Blood Bank Staff
- Monitoring User

Required Features

- Login API
- Logout API
- Protected APIs
- Middleware-based route protection
- Token handling and validation

Bonus Features

- Session/device management
- Rate limiting
- Two-factor authentication concept
- Role & permission handling

---

2. Blood Bag Management Module

Develop complete CRUD APIs for blood bag management.

Blood Bag Fields

Field	Type
Bag Number	String
Blood Group	String
Donor Name	String
Collection Date	Date

Expiry Date	Date
Quantity (ml)	Integer
Status	Enum

Status Types

- Available
- Reserved
- Dispatched
- Expired

---

3. Laravel Relationships (Mandatory)

The project must demonstrate proper usage of Laravel Eloquent relationships.

Suggested Database Structure

BloodBank

- hasMany → Refrigerators

Refrigerator

- belongsTo → BloodBank
- hasMany → TemperatureLogs
- hasMany → BloodBags

BloodBag

- belongsTo → Refrigerator

TemperatureLog

- belongsTo → Refrigerator

User

- belongsToMany → BloodBanks

Relationship Requirements

Candidate must demonstrate usage of:

- Eager Loading
- Nested Relationships
- whereHas()
- withCount()
- Accessors & Mutators
- Query Optimization
- Relationship-based filtering

---

4. Mathematical & Analytical Logic

A. Refrigerator Temperature Risk Analysis

Temperature logs are received every minute.

Temperature Conditions

Temperature	Status
2°C – 6°C	Safe
6°C – 8°C	Warning
Above 8°C	Critical

Required Calculations

For each refrigerator calculate:

- Daily average temperature
- Highest temperature
- Lowest temperature
- Total unsafe minutes
- Risk percentage

Formula

Risk Percentage:

Unsafe MinutesTotal Minutes×100\frac{Unsafe\ Minutes}{Total\ Minutes}\times100Total MinutesUnsafe Minutes×100

B. Blood Expiry Prediction Logic

Implement APIs to identify:

- Blood bags expiring within 24 hours
- Already expired inventory
- Near-risk inventory percentage

---

5. Dashboard APIs

Create APIs for dashboard analytics.

Required Dashboard Data

- Total blood bags
- Available stock by blood group
- Refrigerator health score
- Critical temperature alerts
- Average temperature for today
- Total expired bags
- Active refrigerators

---

6. Advanced Laravel Concepts

Candidate must implement at least THREE advanced Laravel concepts.

Allowed Concepts

- Queues
- Events & Listeners
- Notifications
- Jobs
- Policies

- Repository Pattern
- Service Classes
- API Resources
- Caching
- Custom Helper Functions
- Form Requests
- Observer Pattern

---

7. Alert & Notification System

If refrigerator temperature remains above 8°C continuously for 10 minutes:

System Should

- Trigger critical alert
- Store alert history
- Dispatch notification event
- Queue alert processing

Suggested Laravel Features

- Notification Classes
- Event Broadcasting
- Queue Jobs
- Listeners

---

8. Performance & Scaling Discussion

Candidate should explain how the system can scale efficiently.

Discussion Topics

- Handling 10 million temperature logs per day
- Database indexing strategy
- Query optimization
- Partitioning strategy
- Queue optimization
- Read/Write database separation
- Redis caching

- Failover handling
- Storage optimization

---

9. API Documentation

Provide proper API documentation.

Accepted Formats

- Swagger/OpenAPI
- Postman Collection

Documentation Must Include

- API endpoints
- Request examples
- Response formats
- Authentication method
- Error handling standards

Technical Expectations

Code Quality

The solution should demonstrate:

- Clean architecture
- Reusable code
- SOLID principles
- Proper naming conventions
- Validation handling
- Error handling
- Scalable folder structure

Evaluation Criteria

Category	Marks
Laravel Architecture	20
Relationship Handling	15
Authentication & Security	15
Mathematical Logic	15
Performance Optimization	10
Clean Code Standards	10
Problem Solving Ability	10
Documentation Quality	5

| Total | 100 |

Submission Requirements

Candidate must submit:

- GitHub Repository
- Database SQL Dump
- Postman Collection / Swagger Docs
- README File with setup instructions
- Architecture explanation document

Bonus Challenges (Optional)

Additional points may be awarded for implementing:

- Real-time dashboard
- WebSocket integration
- Docker setup
- Unit & Feature Tests
- Multi-tenant architecture
- RFID simulation APIs
- MQTT integration concepts
- Queue monitoring
- CI/CD pipeline setup

Expected Deliverables

At the end of the assessment, the candidate should deliver:

1. Fully working Laravel backend APIs
2. Proper database structure
3. Secure authentication system
4. Optimized relationship queries
5. Mathematical logic implementation
6. Clean and scalable codebase
7. Documentation and setup guide